



System design

Poll Widget

★ Premium



Medium



30 mins



1.54k done

Report an issue 🛠️

Asked at these companies



Figma

Question

Design a poll widget that can be easily embedded on websites, such as articles and blogs, to allow website viewers to vote on options.

Which JavaScript libraries or frameworks do you prefer?

BEFORE VOTING

☐ React

☐ Vue

☐ Svelte

Choose one or more options to see the results

AFTER VOTING

React

234

Vue

163

Svelte

51

448 total votes · results sorted by popularity

Requirements

- The widget displays the following information:
 - A list of options for users to vote on.
 - The latest number of votes for each option. This is only shown after the user has submitted a vote.

- The following backend APIs are provided:
 - Fetching of the poll results.
 - Record a new vote on an existing poll option.
 - Remove a vote from an existing poll option.
 - It should not require much effort for a website owner to embed a poll widget within their website.
-

Requirements exploration

These are questions you should be asking your interviewer to dive deeper into the problem and refine the requirements.

What are the most critical aspects of the component?

- Ease of embedding the polling widget within websites.
- User experience as a voter.

Does the widget show details of people (e.g. thumbnail) who voted for an option?

Whether we show that level of detail will affect the data model and APIs. We assume a basic version where we only have to show the count.

Can a user vote on multiple options?

Yes, a user can vote on multiple options.

How do we determine the length/proportion of each option bar to render?

You are free to decide that.

In what order should the options be shown? By popularity/user has voted/random?

Popularity.

How are the options determined? Can users add more options?

The poll is created by the website owner in a separate admin portal, and the options are determined during poll creation and cannot be modified afterward.

Is there a maximum number of options shown in the widget?

The maximum number of options is 6.

Must users be logged in on the page in order to vote?

Anyone can vote regardless of whether they are logged in or out. Votes should be persisted for the same user.

Exercise Beta

Before designing a poll widget, which requirements can change its voting, persistence, or embedding architecture? Select all that apply.

- ☐ 1 Clarify first-party versus third-party embedding, isolation and host communication, traffic spikes, and update frequency.
- ☐ 2 Specify optimistic feedback, offline or retry behavior, duplicate prevention, keyboard controls, and result announcements.
- ☐ 3 Choose the result-bar palette before deciding whether votes are authenticated or anonymous.
- ☐ 4 Define single- versus multi-choice voting, authentication, repeat-vote policy, option ordering, and when results become visible.



Reveal answer

Check answer ←

Architecture

The architecture depends on how the widget is delivered to third-party pages and how its components communicate with the host page and the backend.

Rendering approach

We should first evaluate the possible rendering approaches because they will affect the architecture and subsequent discussions.

Generally speaking, there are two approaches for rendering external widgets/components on a page:

- Render in an `<iframe>` (different browser context)
- Render directly within the page (same browser context)

Note that we're talking about how to render the widget, which is different from the distribution approach, meaning how to run the code that renders the widget.

Render in an `<iframe>` (different browser context)

`<iframe>` (inline frame) is an HTML tag on a page that accepts a `src` attribute, which is a URL for a website you want to embed within the host website.

Popular embeddable widgets such as Facebook's Like button, X's embedded posts, YouTube's embedded videos, and Disqus' embedded comments are `iframe`s. They are essentially websites that only render the content to be embedded.

Check out these examples to see for yourself:

Facebook Like Button

Facebook's [Like Button documentation](#) demonstrates both JavaScript SDK and direct `<iframe>` embed options. The live widget is not embedded here because third-party embed behavior can change independently of this article.

Embedded posts ([X Publish](#))

X's supported contract uses generated markup plus the vendor script. Use the official publisher instead of depending on private iframe URLs or feature flags.

Using `iframe`s is a very common technique for embedding your content within third-party websites. In the case of a poll widget, the widget will be the only UI being rendered on the website that's defined as the `iframe`'s `src`.

Pros

- An `iframe` is a separate website and hence a separate **browsing context**. The contents of the `iframe` are isolated from the hosting site and vice versa.
 - The styling of the widget will not be affected by any CSS of the host website.
 - The widget's JavaScript environment is not affected by scripts running on the host website, which can contain polyfills or monkeypatching, affecting runtime behavior in an unpredictable manner.
- Simple to set up because adding an `<iframe>` to the page only involves changing HTML. Users do not need much technical knowledge to achieve that.

Cons

- Needs a web server to host a website that renders the widget. This is not a huge deal because a web server is needed to serve the poll results anyway. Depending on the type of website you build, this setup can range from simple to complex.
- It is slower to load a separate website than to render it directly into the page.
- Because of the isolation, the host website cannot use CSS to customize the component internals.

There are two common ways to get the `<iframe>` on the page, using [Facebook's Like Button developer documentation](#) as an example:

1. Run JavaScript code that dynamically adds an `<iframe>` into the DOM.

JavaScript SDK IFrame



Step 1: Include the [JavaScript SDK](#) on your page once, ideally right after the opening body tag.

```
<div id="fb-root"></div>
<script async defer crossorigin="anonymous"
src="https://connect.facebook.net/en_GB/sdk.js#xfbml=1&version=v15.0"
nonce="rCx0kZk"></script>
```

Step 2: Place this code wherever you want the plugin to appear on your page.

```
<div class="fb-like" data-href="https://developers.facebook.com/docs/plugins/"
data-width="" data-layout="standard" data-action="like" data-size="small" data-
share="true"></div>
```

2. Directly embed the `<iframe>` code into the HTML.

JavaScript SDK IFrame



Place this code wherever you want the plugin to appear on your page.

```
<iframe src="https://www.facebook.com/plugins/like.php?
href=https%3A%2F%2Fdevelopers.facebook.com%2Fdocs%2Fplugins%2F&width=450&layout
=standard&action=like&size=small&share=true&height=35&appId" width="450"
height="35" style="border:none;overflow:hidden" scrolling="no" frameborder="0"
allowfullscreen="true" allow="autoplay; clipboard-write; encrypted-media;
picture-in-picture; web-share"></iframe>
```

The benefit of the JavaScript approach is that the script can customize the iframe rendering (e.g. theme, size) based on the environment if necessary. The direct `<iframe>` approach is simpler but less flexible.

Render within the page (same browser context)

Just like how a script can dynamically inject an `<iframe>` into the DOM, it can also directly add the DOM elements needed to render the poll and the poll results.

Pros

- It is fast to render the poll widget within the same page.

Cons

- The poll widget can be affected by the host website's JavaScript environment and global styling. There's no telling what kind of global styling is present, and it is very likely for the widget's appearance to be affected by the website's CSS.

The question is how to run the third-party JavaScript on the page to achieve the above.

1. Download the script via a CDN.

This approach is similar to Facebook's Like Button JavaScript SDK approach: add a `<script>` tag to the page that downloads and executes some external JavaScript.

2. Distribute the code via `npm`

`npm` is a package manager for JavaScript projects, and the widget code can be packaged into an npm project so that projects can add it as a dependency. Website owners will need to possess the technical knowledge of how to add a new `npm` package to their `package.json`.

Distributing only via `npm` is not a great idea because not all websites are built using a JavaScript-based build system. Low-code websites like WordPress, Webflow, and Blogger do not allow for including third-party JavaScript code on the page via npm.

Which rendering approach is better?

For embedding widgets, the `<iframe>` approach is clearly superior due to the encapsulation of styles and environment that an `iframe` provides.

✓ Default to `iframe` embedding for third-party widget distribution

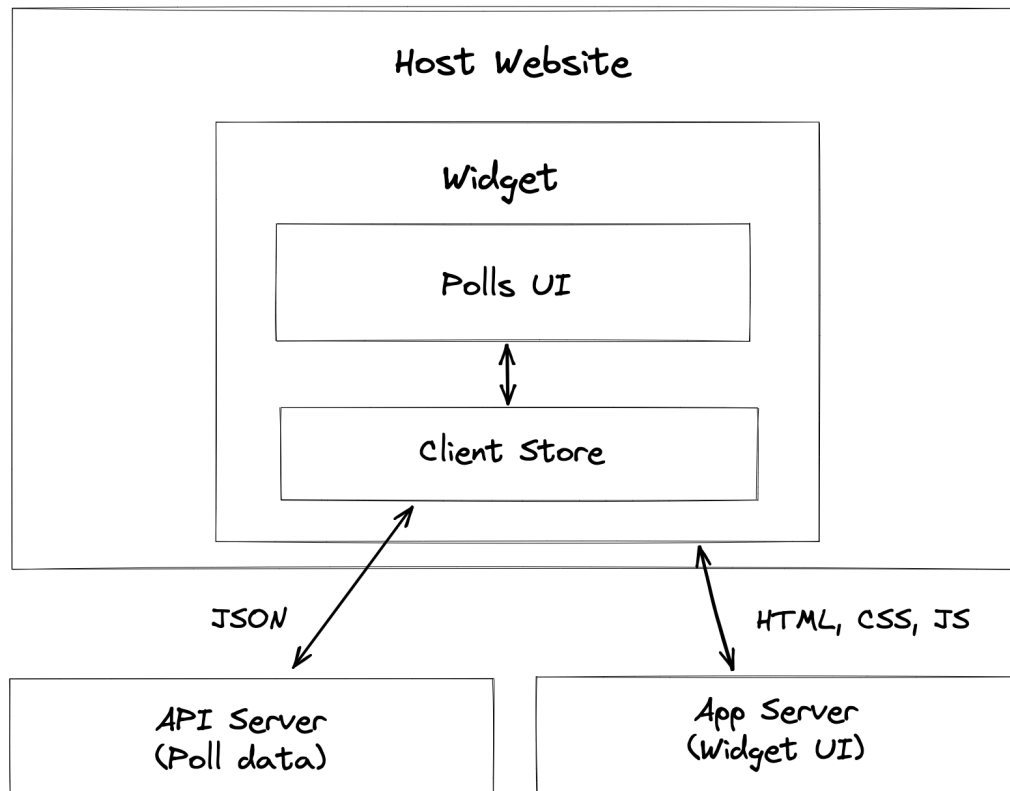
Same-context rendering exposes the widget to unpredictable host CSS and JavaScript (polyfills, monkeypatching, global styles), and there is no practical way to guard against every host environment. `Iframe` isolation is what makes the widget behave the same across every site that embeds it, which is the whole point of the product.

Which distribution approach is better for rendering `<iframe>` s?

Embedding `<iframe>` s directly in the HTML is the easiest, and the upsides of using a `<script>` -based approach are not that significant. That said, it'd be nice to give developers the option to choose between the two approaches. Facebook and YouTube provide both JavaScript SDKs and direct `<iframe>` embed options.

The discussion below will assume an `<iframe>` embed approach.

Diagram

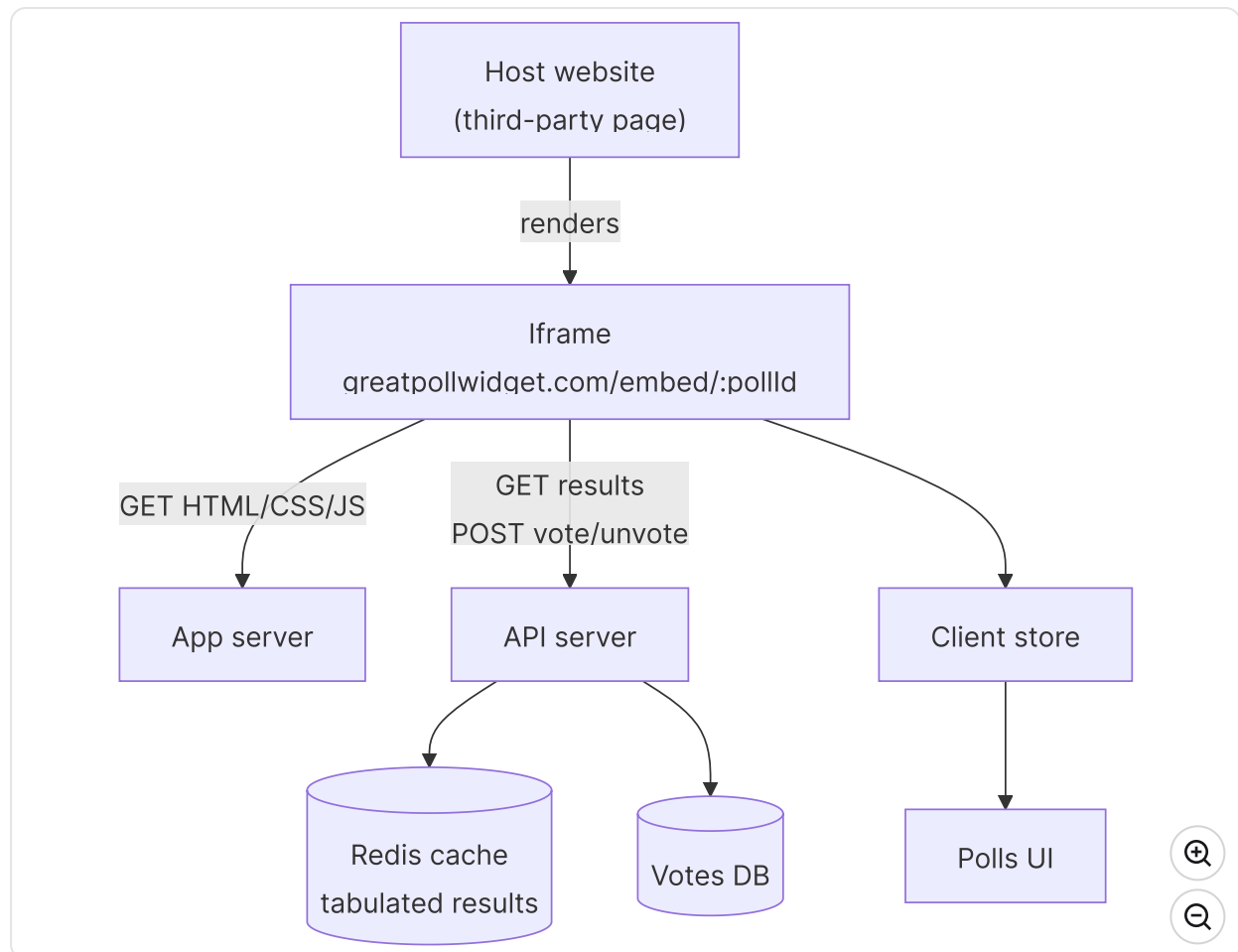


Component responsibilities

- **Host Website:** Host website that embeds the widget as an `<iframe>`.
- **App Server:** Renders the widget UI as a website by serving the HTML, CSS, and JavaScript needed.
- **API Server:** Server that returns poll result data and accepts votes/unvotes. Poll creation remains in the separate admin product.
- **Client Store:** The module that interacts with the API server and stores data for the UI.
- **Polls UI:** UI for the poll options.

*Note: The **App Server** and **API Server** have been split into separate components for clarity, but they can share one deployment. Browser behavior still depends on their exact origins.*

The relationships between these pieces can be summarized as a flow of responsibilities, from the host site embedding the iframe, to the app server serving the widget document, to the API server returning tallied poll data.



Iframe embed topology across host site, app server, and API server

Exercise 1 of 3 Beta

A poll widget must behave consistently on third-party pages whose global CSS and JavaScript may be unknown or modified. Which rendering boundary best isolates both the widget's styles and runtime environment?

- 1 Serve the poll inside an iframe with its own document and JavaScript environment.
- 2 Render into a shadow root while running the widget code in the host page's JavaScript context.
- 3 Render into the host DOM with prefixed class names and a widget-wide CSS reset.
- 4 Require every publisher to remove global styles and polyfills around the widget container.



Reveal answer

Check answer ←

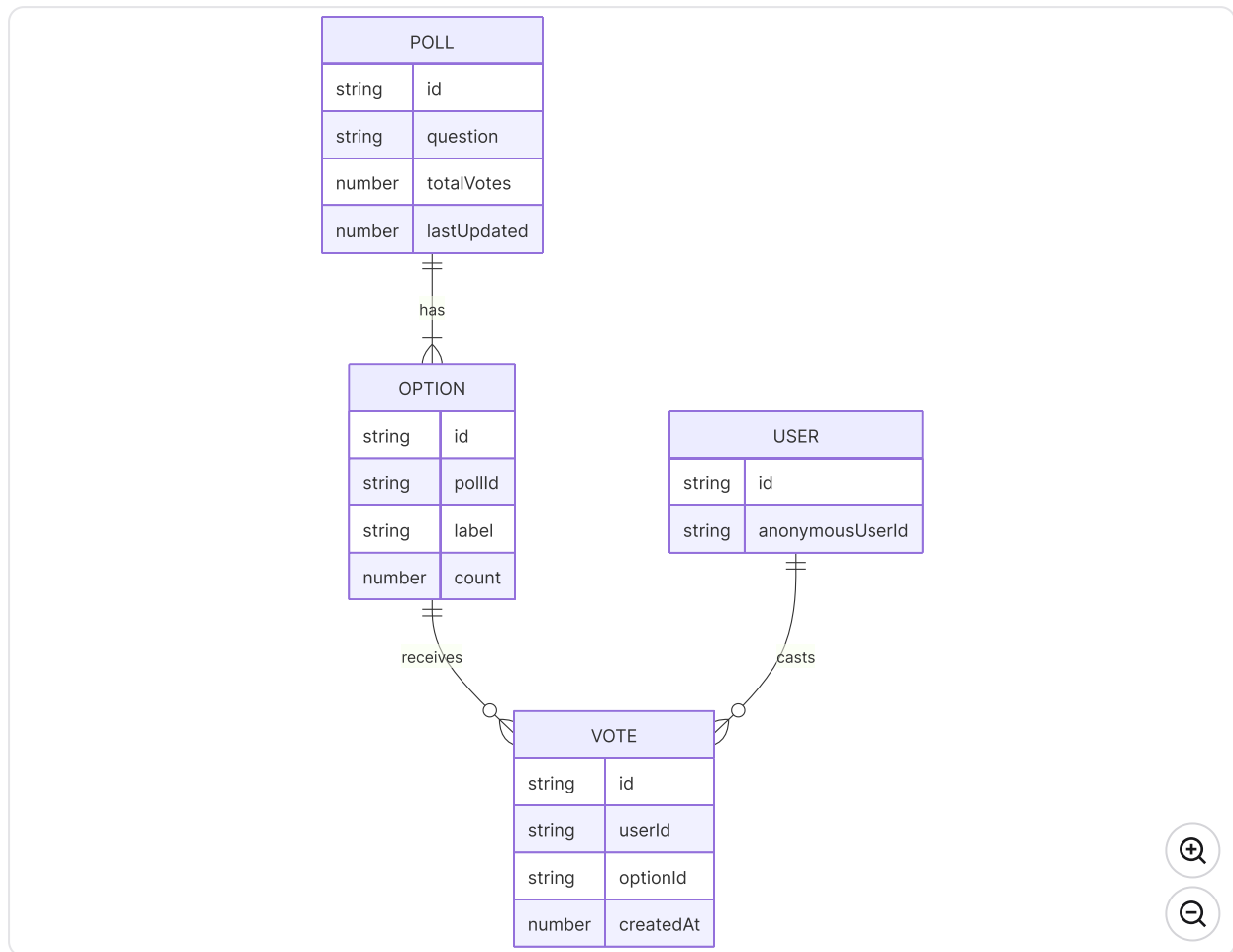
Data model

The data model for a polling widget is quite simple. Something like this will be sufficient:

```
1  const state = {
2    lastUpdated: 1628634891,
3    // Number of respondents. In a multi-select poll, option counts can ad
4    // to more than this value.
5    totalVotes: 421,
6    question: "Which is your favorite JavaScript library/framework?",
7    options: [
8      {
9        id: 123,
10       label: "React",
11       count: 234,
12       userVotedForOption: false,
13     },
14     {
15       id: 124,
16       label: "Vue",
17       count: 183,
18       userVotedForOption: true,
19     },
20     {
21       id: 125,
22       label: "Svelte",
23       count: 51,
24       userVotedForOption: true,
25     },
26     // ...
27   ],
28   // Client-only state initialized from userVotedForOption.
29   selectedOptions: [124, 125],
30 };
```

The option flags are the last server-confirmed selection. `selectedOptions` is client-only state initialized from those flags and updated as the user interacts.

On the server side, the same data is backed by a few related entities: the poll itself, its options, the votes cast by users, and the relationship between a viewer and the options they have selected.



Server-side poll entities and viewer-vote relationship

Exercise Beta

A poll receives concurrent result updates and may be embedded more than once on a page. Which values belong in its logical data model? Select all that apply.

- ☐ 1 Namespace poll state by the host element's CSS selector so each embed can restore its selection.
- ☐ 2 A stable poll ID and ordered options with stable IDs.
- ☐ 3 Server-originated counts or totals plus a version or timestamp used for reconciliation.
- ☐ 4 Persist the displayed rounded percentages and derive counts from them when a live update arrives.

5

The current viewer's selected option IDs.

 Reveal answer Check answer ↩

Interface definition (API)

There are three kinds of APIs to discuss:

- **Embed API:** How websites should embed the `<iframe>`.
- **Components API:** How to build the poll widget and the props the components accept.
- **Server APIs:** The HTTP APIs to fetch results, record new votes, and remove votes.

Embed API

Websites should be provided with some code to be copied and pasted to render the poll widget. The `iframe`'s `src` attribute should be a unique URL specific to a poll instance.

```
1 <iframe
2   src="https://greatpollwidget.com/embed/{poll_id}"
3   title="Poll widget for your favorite JavaScript framework"
4   style="border: 0; height: 200px; width: 100%; max-width: 450px"
5 ></iframe>
```

Use CSS on the `iframe` element to remove its border and size it to the widget. The embedded document should use its own CSS, such as `html, body { margin: 0; }`. The `frameborder` and `scrolling` attributes are obsolete and should not be included in new embed code.

For widgets below the initial viewport, add `loading="lazy"` to the `<iframe>` so the browser can defer loading until the widget is near the viewport. Keep the default eager loading for a poll that should be available immediately when the page opens.

This fixed height can clip longer polls or translations. If dynamic sizing is needed, the iframe can be resized using `ResizeObserver` and validated `postMessage` messages; otherwise, retain scrolling as a fallback.

Components API

- `Poll`
 - Server URL
- `PollOptionList`
 - Question
 - Maximum number of options displayed
- `PollOptionItem`
 - ID
 - Label
 - Number of votes for an option
 - Selected state
 - Event handler: `onSelectionChange`

```
1 // Example code in React
2 <Poll serverUrl="https://greatpollwidget.com/api/{poll_id}">
3   <PollOptionList question={question}>
4     {options.map((option) => (
5       <PollOptionItem
6         key={option.id}
7         id={option.id}
8         label={option.label}
9         count={option.count}
10        isSelected={selectedOptions.includes(option.id)}
11        onSelectionChange={(isSelected) => {
12          updateSelectedOption(option.id, isSelected);
13        }}
14      />
15    ))}
16   </PollOptionList>
17 </Poll>
```

`PollOptionItem` reports the checkbox's next selected state. The parent updates `selectedOptions` and calls the vote or unvote API accordingly.

Server APIs

The server APIs are ideally served from the same origin as the iframe so that no CORS configuration is needed, e.g. `https://greatpollwidget.com/api/{poll_id}/results` and `https://greatpollwidget.com/api/{poll_id}/submit`. If the scheme, host, or port differs, configure CORS and credentials explicitly.

Fetching results

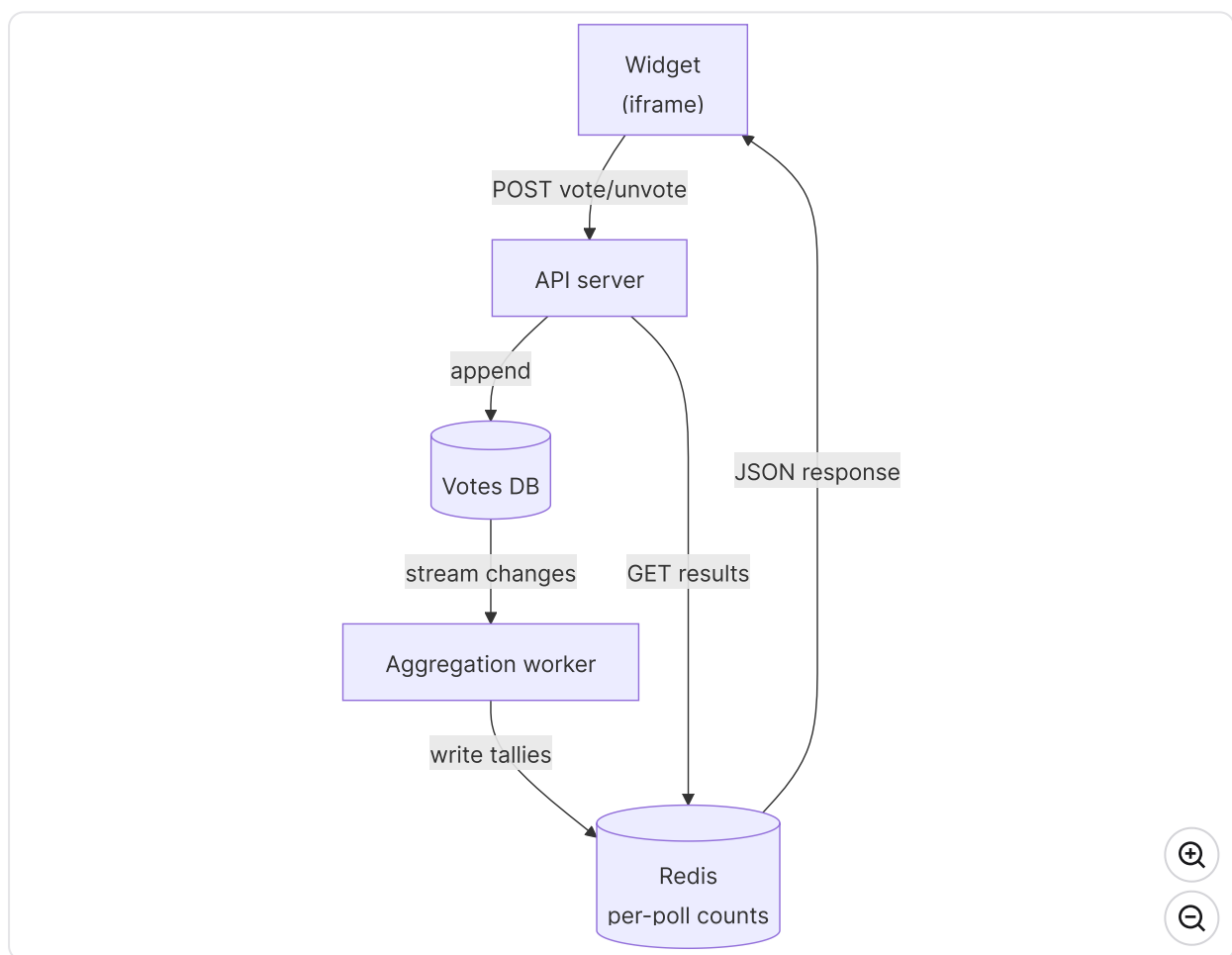
What format should the API return the results in?

1. Return the options and the counts for each: The server returns something like:

```
1  {
2    "totalVotes": 421,
3    "question": "Which is your favorite JavaScript library/framework?",
4    "options": [
5      {
6        "id": 123,
7        "label": "React",
8        "count": 234,
9        "userVotedForOption": false
10     },
11     {
12       "id": 124,
13       "label": "Vue",
14       "count": 183,
15       "userVotedForOption": true
16     },
17     {
18       "id": 125,
19       "label": "Svelte",
20       "count": 51,
21       "userVotedForOption": true
22     }
23   ]
24 }
```

- Pros
 - Client does not need to do tabulation of results, simply render the results.
 - Payload is small, contains exactly the data that needs to be displayed.
- Cons
 - Server has to do the processing, but this is likely better because the server can cache/memoize the results, especially for popular polls, and return the cached results for different users.
 - The server will need to update the tabulation every now and then. This is a good use case for in-memory key/value stores like Redis/Memcached.

The tabulation pipeline that backs this API shape typically separates the write path (incoming votes) from the read path (cached tallies) so that hot-poll reads stay fast regardless of write volume.



Vote tally aggregation pipeline

2. Raw list of poll responses: The server returns something like:

```
1  {
2    "question": "Which is your favorite JavaScript library/framework?",
3    "options": [
4      {
5        "id": 123,
6        "label": "React"
7      },
8      {
9        "id": 124,
10       "label": "Vue"
11     },
12     {
13       "id": 125,
14       "label": "Svelte"
15     }
16   ],
17   "votes": [
18     {
19       "optionId": 123,
20       "createdAt": 1628634891
21     },
22     {
23       "optionId": 123,
24       "createdAt": 1628634892
25     },
26     {
27       "optionId": 124,
28       "createdAt": 1628634893
29     }
30   ]
31 }
```

- Pros
 - Can do all the tabulation entirely on the client side, and hence sorting/filtering does not incur any network latency.
- Cons
 - Doesn't scale well when there are lots of responses; the network payload will be huge.
 - The client needs to do the tabulation of votes, which can be expensive on low-end devices and when there are many votes.

- Individual timestamps expose behavioral data the widget does not need.
- Different clients can derive inconsistent tallies from the same event stream.

Which to choose?

Directly returning the counts is usually the superior option, as there is typically no need to do tabulation or manipulation of the data on the client, which does not scale for popular polls with tens of thousands of votes. It also avoids exposing individual response timestamps.

Submitting votes and unvoting

The vote submission/unvoting APIs can accept a list of option IDs and an idempotency key that is reused for retries, then return the updated poll results in a similar format to the poll results fetching API.

Exercise 1 of 5 Beta

An embedded poll can grow when labels wrap or translations are longer than expected. Which parts belong in a safe dynamic-height design? Select all that apply.

- 1 Keep scrolling available when dynamic measurement or cross-window messaging does not run. ☐
- 2 Observe the embedded document's content size and send height changes to the host page. ☐
- 3 Have the host verify the message origin, source window, and expected payload before applying a height. ☐
- 4 Set the parent iframe's height directly from CSS inside the embedded document. ☐

[Reveal answer](#)[Check answer ↩](#)

Optimizations and deep dive

This section covers the details that make an embeddable poll reliable, understandable, fast, and accessible on third-party pages:

- **Persisting votes across sessions**: This section explains how anonymous voters can be recognized across reloads and repeat visits.
- **Rendering the poll options**: This section discusses ways to render result bars and option states without losing clarity or accessibility.
- **User experience**: This section covers hidden results, vote changes, loading states, confirmation, and other poll lifecycle details.
- **Performance**: This section explains how to keep an embedded widget lightweight, fast to initialize, and cheap to render.
- **Accessibility**: This section covers text equivalents for visual results, keyboard interaction, and assistive-technology behavior.
- **Network**: This section discusses out-of-order vote responses, retries, and request handling for quick vote or unvote actions.
- **Internationalization (i18n)**: This section covers localized widget strings, creator-provided content, and iframe language configuration.

Persisting votes across sessions

Because anyone can vote without having to log in first, we will need a way to identify users across sessions; otherwise, the user will not see that they have already voted after the browser tab is closed.

We can use a cookie containing a randomly generated anonymous user ID during the initial load if there is no existing ID cookie.

This helps the poll widget identify returning users and show the options they have already voted for. Because the widget runs in a third-party iframe, some browsers may block or partition this cookie, so anonymous continuity is best-effort. Stronger continuity requires partitioned storage or authenticated identity, and the same identifier should not be reused across publisher sites.

Note that users can work around this by using different browsers or different devices. The only way to prevent this sort of abuse is to have user authentication.

! An anonymous cookie is not a vote-stuffing defense

An anonymous ID cookie only keeps the same browser from accidentally voting twice. It is trivial to bypass by clearing cookies or switching browsers. Treat this as UX state for returning voters, not as an integrity control; authentication, rate limiting, and server-side checks can be discussed if abuse prevention is in scope.

Exercise 1 of 2 Beta

An embedded poll allows anonymous voting and should restore server-confirmed choices for the same browser when possible, without claiming to identify a person. Which persistence design best fits?

- 1 Derive a stable identity from fonts, screen properties, and network attributes, then restore votes by that fingerprint.
- 2 Store the selected option IDs only in local storage and treat them as the authoritative vote record after reload.
- 3 Store a random anonymous ID in a cookie and associate that ID's confirmed option selections with the poll on the server.
- 4 Ask the publisher page to retain the selected option IDs in a hidden field and return them whenever the iframe reloads.



Reveal answer

Check answer ↩

Rendering the poll options

The polling results consist of multiple bars of various widths, and there are multiple ways to render such a result. It's worth discussing the various ways of rendering bars of varying widths.

What a full bar represents

A full bar can have two common representations:

1. Full width represents 100% of all responses. If an option is X% of the total, it will take up X% of the container width.
 - Accurate representation of the proportion of options.

- Bad if the ratio is very even and there are many options with low %. Hard to discern because many bars will be very short.
2. The top-voted option is rendered with the full width, and other options are a proportion of it. E.g. the top-voted option is 40% of the total votes but will take up the entire container width. If another option is 20%, it will take up half the width.
 - Useful for highlighting the relative difference between the options.
 - Might give viewers a false impression that the top-voted option has a higher ratio than it actually does.

The first option is the more common one and is used by Reddit and X.

Rendering bars of dynamic widths

The number of votes for an option over the total number of votes will be the proportion of the full width to render the bar. E.g. 400 votes for an option out of a total of 1000 votes will mean that the bar should be rendered at 40% of the full width. Here, `totalVotes` means the number of respondents, so percentages in a multi-select poll can add up to more than 100%. A poll with no respondents renders empty bars without dividing by zero. Since there are infinitely many possible values for the width, it is not practical to use static CSS classes to render a bar for a specific width. A better way would be to use inline styles that are dynamically generated during rendering.

1. Using CSS `width` inline style: This is the most common approach, and the only small downside is that if there's a need for animating the bars expanding/contracting, animating the `width` property is slower than `transform`. However, the widget is mostly static, so the animation issue is largely not present.

```
<div style="width: 40%">React</div>
<div style="width: 30%">Vue</div>
```

2. Using `transform: scaleX()` style: This method involves scaling the element horizontally.

```
<div style="transform: scaleX(0.4); transform-origin: left">React</div>
<div style="transform: scaleX(0.3); transform-origin: left">Vue</div>
```

Note that `scaleX()` will also transform the contents within it and make them appear horizontally squished.

We should size each bar relative to the full width of the widget instead of using a hardcoded pixel value calculated when the page loads. This allows the bars to adapt when the widget is resized. With `width`, use a percentage; with `transform: scaleX()`, use a unitless scale factor between `0` and `1`.

If we are fine with some precision loss, then we could have 101 class names for percentages from 0 to 100. But in general, this is not a great approach, and inline styles are the preferred approach.

Exercise 1 of 3 Beta

A poll result says the leading option was selected by 40% of respondents. The chart should communicate absolute respondent share rather than only the gap between choices. What should a full-width bar represent?

- 1 Treat 100% of respondents as full width and size each option by its count divided by respondent count.
- 2 Use the widest label as the full-width reference so text and bars remain aligned.
- 3 Treat the sum of all option selections as full width, even when respondents may choose several options.
- 4 Treat the highest option count as full width and scale every other option relative to it.

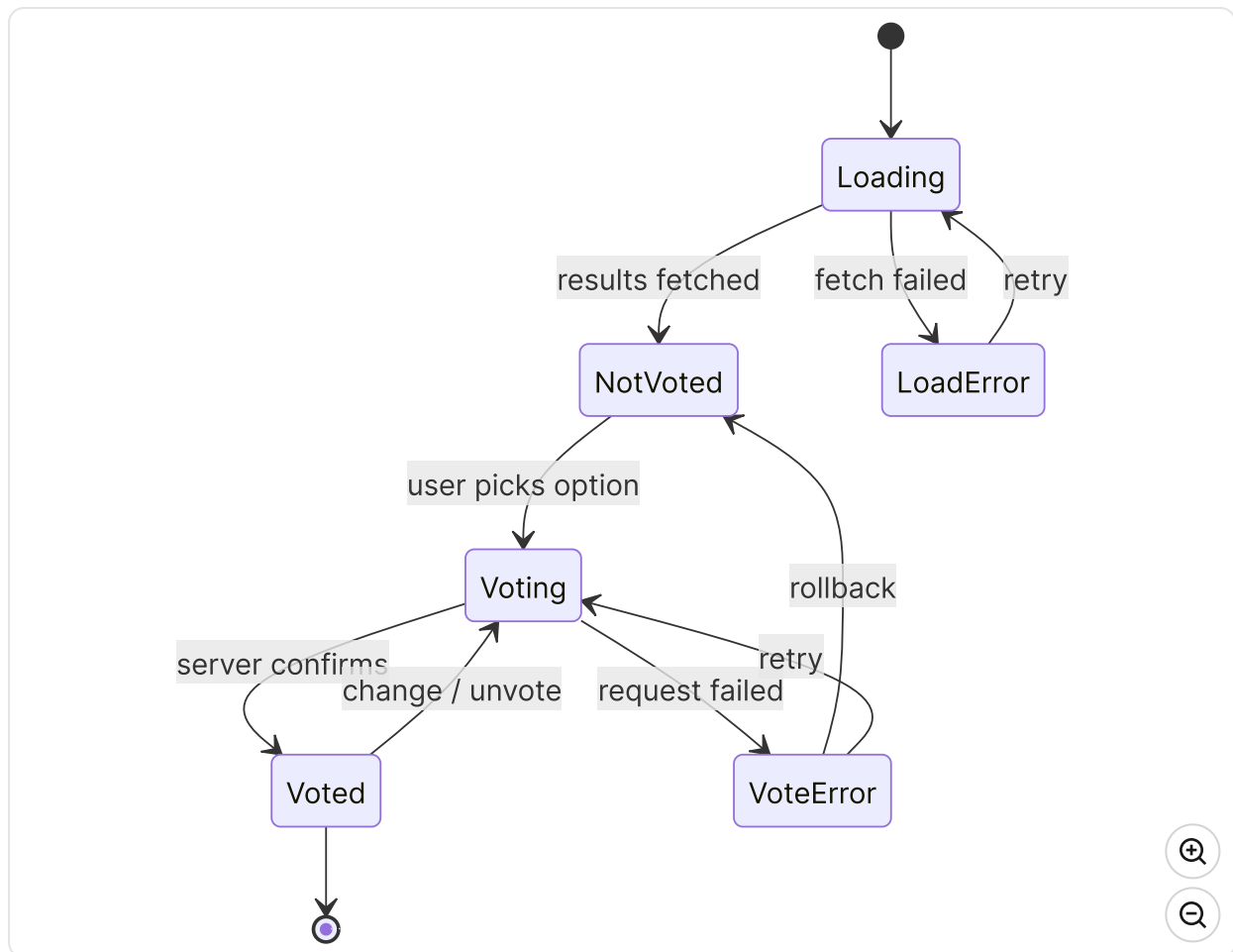


Reveal answer

Check answer ↩

User experience

The widget moves through a small set of visible states over its lifetime, and the UX guidelines below are best understood against that lifecycle: when results are shown, when they are hidden, and how the user can unvote, change their vote, or recover from errors.



Poll widget lifecycle and vote states

- When the poll is still loading, use a stable-size skeleton in the shape of bars to hint that this is a poll and reduce layout shift. Keep it static when the user prefers reduced motion.
- Poll results should be hidden before the user has voted, to reduce bias.
- Consider having a "See responses" function for people who just want to see the results without voting.

Exercise 1 of 2 Beta

An embedded poll loads its initial data, lets an unvoted viewer respond, and may encounter either a fetch failure or a vote failure. Which UX decisions make those states clear without biasing the vote? Select all that apply.

1

Reveal cached result bars beneath the skeleton before the viewer chooses an option.

☐

2

Provide an explicit way to view results without casting a vote.

☐

3

Keep counts and result bars hidden until the viewer votes or explicitly asks to see responses.

☐

4

Return to the initial unselected view after any failure without explaining what happened.

☐

5

Reserve the widget's shape with a poll-like skeleton and keep it static when reduced motion is preferred.

☐



Reveal answer

Check answer ↩

Performance

Because the widget is embedded on third-party pages, every kilobyte and millisecond matters. The subsections below cover fast initial rendering and keeping the bundle small.

Fast rendering

As mentioned earlier, to achieve fast rendering of the results, the server API should return the tabulated results, not raw results, so that the client does not need to do any tabulation of results.

Prefer server rendering for the initial iframe document, but preserve the product rule: an unvoted viewer receives the question, options, and stable pending shell without visible tallies.

✓ Server-render the unvoted shell, not private results

Canonical tallies can be cached independently, then overlaid with the private viewer selection and visibility decision before rendering.

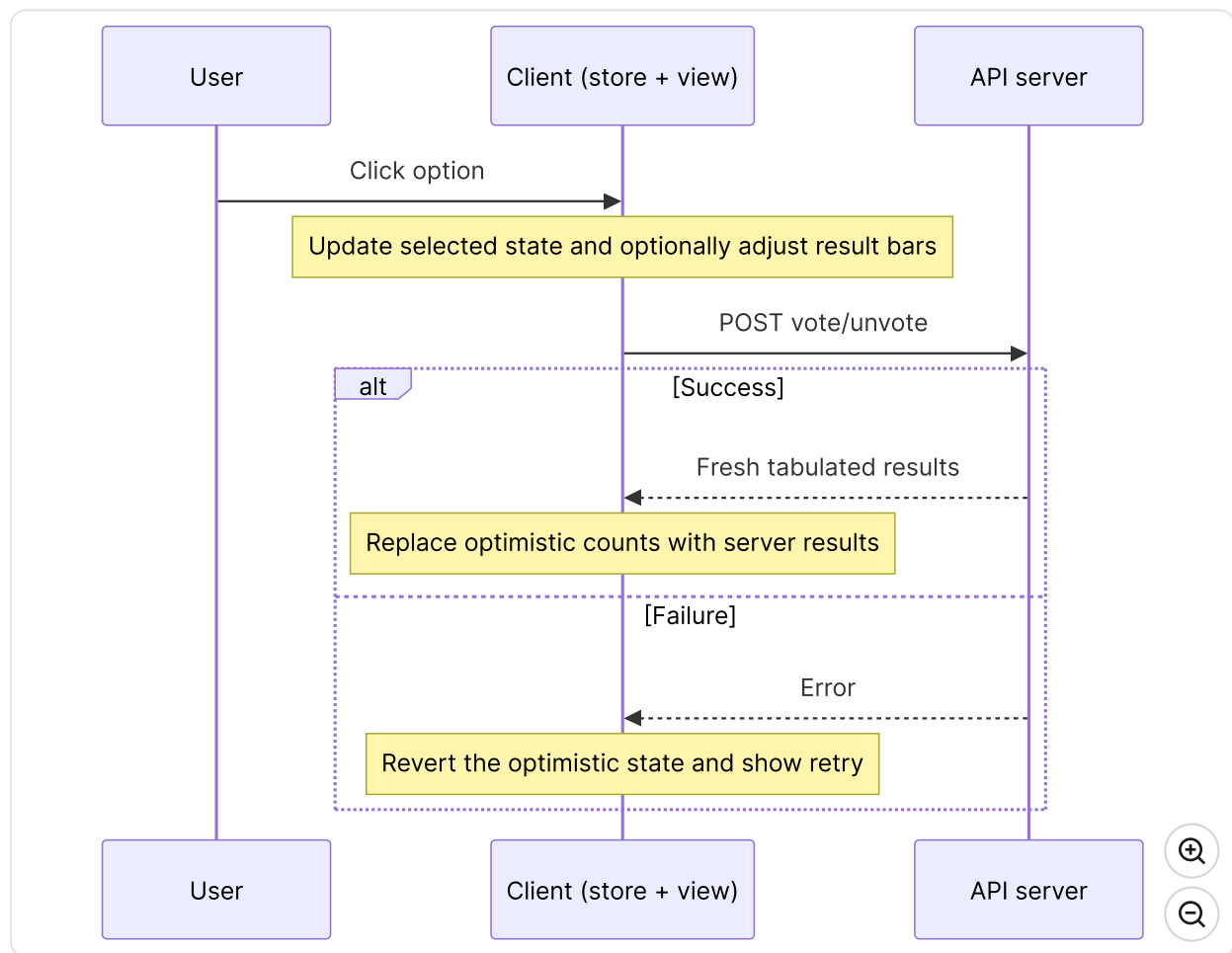
Fast interactions via optimistic updates

Optimistic updating is a technique where the browser shows the new UI state after a server request is fired, even before a response is received from the server. Because the client has the current results during initial load, it can also increment the newly voted option and compute new bar ratios locally.

However, there are a few caveats:

- The selected state should update immediately for every poll. Optimistically changing aggregate bars is a separate optional enhancement and is most useful when one response creates a visible difference.
- Client-side counts can be stale on a popular poll because other people may vote before the server responds.

In most cases, the client can render the optimistic state first and then render again with the latest results from the server.



✓ Separate interaction feedback from aggregate-bar updates

Immediate selected-state feedback matters even when one vote cannot visibly move a result bar. Speculative bar changes are optional, and the server response remains authoritative.

Scalability issues

Given that there are only a maximum of 6 options, we will not run into the issue of rendering too many options or creating a large DOM. A future product with many choices would need a different interaction, such as search, grouping, or pagination.

Exercise Beta

A popular poll's aggregate tallies are identical for many viewers, but whether results are visible and which options the viewer selected are private to that request. Which caching boundary preserves both fast reads and correct viewer state?

- 1 Cache result-bearing markup for all viewers and use client-side CSS to hide its tallies when the viewer has not voted.
- 2 Disable caching for every poll response because the selected-option state makes all aggregate data viewer-specific.
- 3 Cache the public per-poll tallies, then combine them with the viewer's confirmed selections and visibility decision before producing the response.
- 4 Cache the first fully rendered response for the poll, including that viewer's selected options, and serve it to later viewers.

[Reveal answer](#)[Check answer ↩](#)

Accessibility

A poll is highly visual, so accessibility work centers on giving assistive tech a text-equivalent story and making the interactive elements keyboard operable.

Screen readers

Polling widgets are inherently very visual UI elements, and we need to take special care to ensure users relying on screen readers can still understand what is being shown on the screen.

- Put the question in a `<legend>` and render each multi-select option as a labeled native checkbox. Keep counts and percentages as visible text, using `aria-describedby` only for additional explanation.

- Use a separate polite status region for pending, submitted, and failed states rather than announcing every tally change.

Match ARIA roles to the poll's selection model

Use radiogroup/radio only for single-select polls. Multi-select polls (like the one assumed in this design) should use checkbox semantics instead, because radio groups imply mutually exclusive selection and will confuse screen reader users when more than one option can be chosen.

Keyboard interaction

- Native checkboxes provide Space-key behavior, focus, state, and disabled semantics without recreating them on a `div`.

Exercise Beta

A poll supports single- and multi-choice modes and updates result bars after submission. Which behaviors make the complete flow operable and understandable? Select all that apply.

1 Group related radio buttons or checkboxes with a clear poll question. ☐

2 Use clickable divs with `tabindex="0"` and handle Enter, but keep selected state only in CSS. ☐

3 Announce the submitted or failed state without repeatedly interrupting for every count change. ☐

4 Use native inputs with programmatic labels and visible focus. ☐

5 Add accessible option labels but expose each result percentage only through its visual bar width. ☐



Reveal answer

Check answer ↩

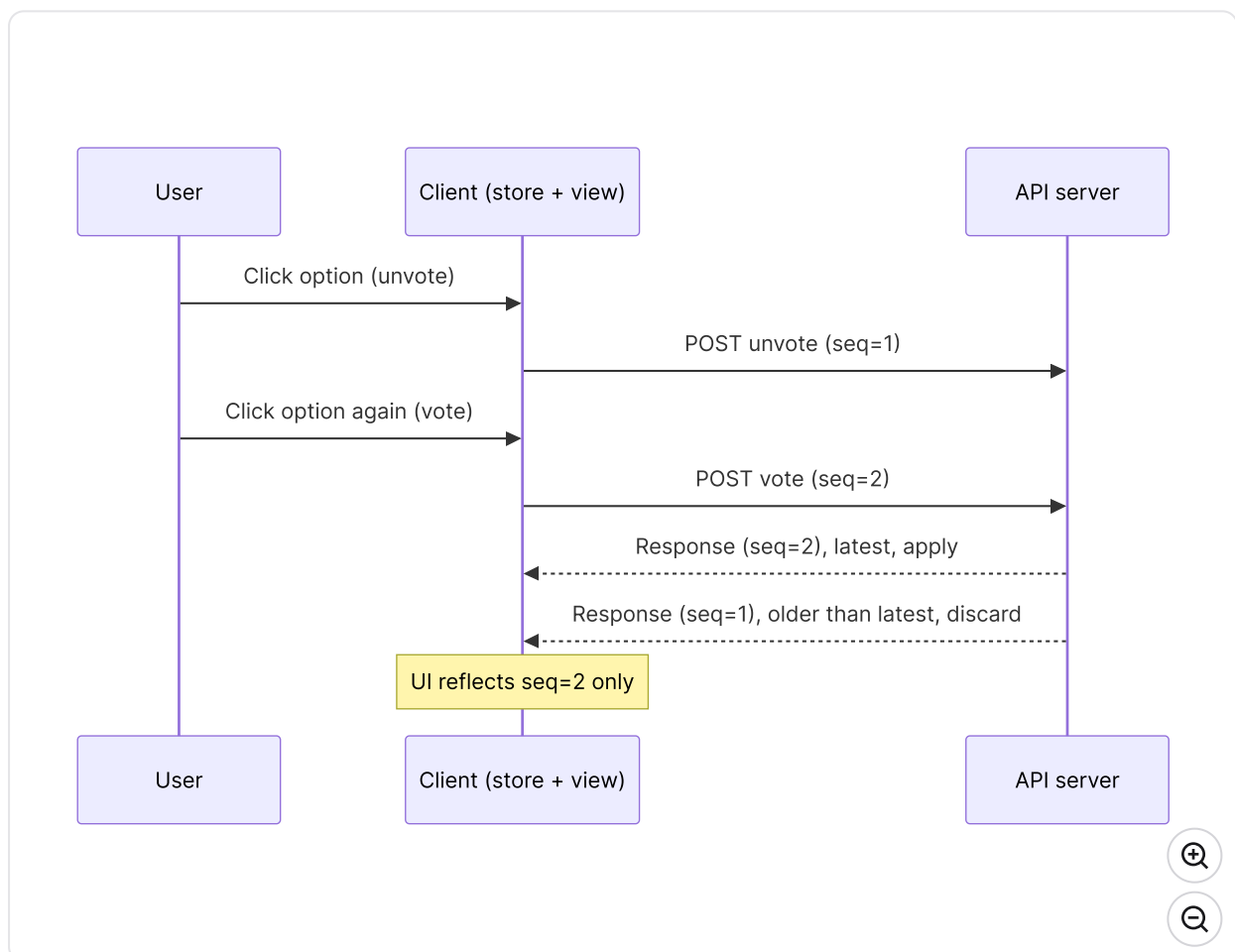
Network

- Request responses could be out of order if someone votes/unvotes in quick succession.
 - Keep track of the latest response and ignore older responses when rendering.
- Show errors in the UI if the API submission fails.

! **Out-of-order vote and unvote responses can corrupt the displayed tally**

If a user toggles the same option quickly, an older response can arrive after the newer response and revert the UI. Tag each in-flight request with a monotonically increasing sequence number (or track the latest request ID per option) and ignore any response that is not the newest.

The sequence below illustrates how a monotonically increasing request ID lets the client ignore a stale unvote response that arrives after a newer vote response.



Ignoring stale UI responses during rapid vote and unvote toggling

The idempotency key from the vote API lets the server deduplicate transport retries of one logical mutation. If the interviewer asks about ordering distinct vote and unvote

mutations on the server, discuss versioning or mutation sequencing as a separate follow-up.

Exercise Beta

A viewer quickly unvotes and then votes for the same option. The first response arrives last, and one request is retried after a timeout. Which client behaviors prevent double counting and stop the older response from replacing the newer UI state? Select all that apply.

- ☐ 1 Apply each response as it arrives because the requests were sent in user-action order.
- ☐ 2 Keep one pending vote record but increment the visible result again whenever its request is retried.
- ☐ 3 Reuse the same idempotency key when retrying one logical vote.
- ☐ 4 For the latest successful request, replace optimistic counts and selections with the server's canonical results.
- ☐ 5 Tag requests per option and apply a response only if it belongs to the latest request for that option.

[Reveal answer](#)[Check answer ↩](#)

Internationalization (i18n)

If there's a need for i18n of strings in the poll, especially the strings not from the poll creator (e.g. `aria-label`s), the `iframe` embed URL can accept a query parameter for the language, and it'll be up to the website owner to provide the right language.

Exercise Beta

An `iframe` poll is embedded by publishers serving different languages. The poll question and option labels come from the poll creator, while the widget owns controls and accessible labels. Which localization responsibilities belong to the embed contract? Select all that apply.

1

Read the publisher page's `lang` attribute directly from inside the `iframe` and use it as the only locale signal.

☐

2

Translate widget-owned text such as buttons, status messages, and `aria-label`s for the requested locale.

☐

3

Automatically translate the creator's poll question and option labels inside the widget.

☐

4

Let the publisher specify the widget language through the `iframe` URL.

☐



Reveal answer

Check answer ←

Summary

The defining constraint is that the widget runs inside arbitrary third-party pages. The publisher's CSS, JavaScript, and analytics are outside the widget's control, so the first design decision is isolation. That is why the default distribution format is an `<iframe>` served from a poll-specific URL rather than a script that injects markup into the host DOM.

The `<iframe>` embed separates delivery from data. The App Server owns the `iframe` document and can server-render the unvoted question and options, while the API Server handles tabulated counts and vote/unvote mutations behind a Redis cache of per-poll totals. The two read and write paths can then scale independently, and the component surface—a `Poll` containing a `PollOptionList` of `PollOptionItem` children with an `onSelectionChange` callback—remains coherent whether a publisher chooses the `iframe` or scripted distribution path.

Isolation keeps the client model and vote API flat. Since the `iframe` owns its own runtime, the client can keep `question`, `totalVotes`, options carrying `count` and `userVotedForOption`, and a client-only `selectedOptions` array derived from those flags. Server-side, that maps to `POLL`, `OPTION`, and `VOTE` entities keyed by a `USER` identified through a random anonymous user ID cookie. Returning pre-tabulated counts rather than raw vote lists keeps popular polls cheap to read, and returning updated

results from vote/unvote requests gives the client authoritative values to reconcile against.

The frame boundary dictates reconciliation and accessibility. Optimistic selected-state feedback makes a vote feel instantaneous, while the result bars reconcile to the server response. A monotonically increasing request ID prevents stale responses from clobbering newer UI. Because nothing outside the iframe can be relied on, the accessibility story is carried inside the widget with a fieldset, legend, native checkboxes, visible result text, and a polite status region. In an interview, start with the iframe isolation decision; every other choice here follows from it.

References

The references below group poll-widget sources by the topic they inform, so it is easier to map each source back to the part of the article it relates to: production embed examples, iframe and cross-origin messaging, security considerations, and related system design articles.

Production embed examples

This subsection collects real third-party embed products whose iframe and script patterns inform the poll widget's embedding model.

- [Facebook like button](#) for a mature script-plus-iframe embed that hosts interactive state across origins.
- [X Publish](#) for the supported generator for embedded posts.
- [Disqus universal embed code](#) for a widely deployed comments embed, useful as a model for configuration and theming of the poll widget.

Iframe embedding and cross-origin messaging

This subsection covers the browser mechanics behind embedding the widget into third-party sites and communicating with the host page.

- [iframe](#) | [MDN](#) for the element that isolates the widget from the host page's DOM and scripts.

- [Window.postMessage\(\) | MDN](#) for the cross-origin messaging primitive used to coordinate height, theme, and events between the widget and its host.

Security considerations for embeds

This subsection covers the web platform features that keep third-party embeds safe for both the host page and the widget origin.

- [Content security policy \(CSP\) | MDN](#) for controlling which origins can embed the widget and which resources it may load.
- [Cross-origin resource sharing \(CORS\) | MDN](#) for the rules that govern API calls made from the embedded widget back to its backend.
- [SameSite cookies | MDN](#) for how cookie scoping affects user identification and vote-dedup for an embedded widget.

Related system design articles

The articles below overlap with the poll widget on social interactions and third-party embedding.

- [News feed \(Facebook\)](#) for the surrounding social surface where embeddable widgets like polls are often consumed and rendered at scale.

Exercises

Exercise 1 of 21 Beta

Before designing a poll widget, which requirements can change its voting, persistence, or embedding architecture? Select all that apply.

1

Specify optimistic feedback, offline or retry behavior, duplicate prevention, keyboard controls, and result announcements.

☐

2

Clarify first-party versus third-party embedding, isolation and host communication, traffic spikes, and update frequency.

☐

3

Define single- versus multi-choice voting, authentication, repeat-vote policy, option ordering, and when results become visible.

☐

4

Choose the result-bar palette before deciding whether votes are authenticated or anonymous.

☐

Reveal answer

Check answer ↩

Discussions

↶ ↷ | Normal T_T | B I U <> Aa

Type comment here...

0/1000

Post



Report an issue



✓ Mark complete



Subscribe to our newsletter

Notify me

Practice

Get started

JavaScript functions

User interface coding

Study Plans

1 Week Plan

1 Month Plan

3 Months Plan

System design

Quiz

Guides

Front End Interview Playbook

Front End System Design Playbook

React Interview Playbook

Behavioral Interview Playbook

Company

Pricing

Promotions

Roadmap

About

Team

Contact us

Become an affiliate

Careers

Blog

Medium

DEV Community

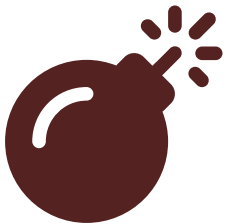


English (US) ▾

© 2026 Codeney Pte Ltd. All rights reserved.

• [Privacy Policy](#)

• [Terms of Service](#)



Syntax error in text
mermaid version 11.14.0